

tutorialverse: Interoperability and quality infrastructure for R tutorials

Shaurita D. Hutchins 
University of Alabama at Birmingham

Samuel Bharti 
University of Alabama at Birmingham

Executive summary

Instructors and learners choosing an R tutorial often have to inspect its repository to find prerequisites and evidence that the exercises still work. Descriptions differ across formats and may leave maintenance status unclear.

tutorialverse will help instructors and learners find R tutorials suited to their needs and see the reported prerequisites, check results, and review status. Authors will describe tutorials with shared metadata. An R package will use adapters to run supported checks through existing engines and produce common reports. A searchable registry will combine descriptions, reports, and human reviews, linking readers to original lessons. Community review and scheduled checks will help keep entries current.

An [initial prototype](#) already demonstrates the proposed adapter model across all three engines. Grant work will extend this foundation to curated registry records, CI-based validation, author-effort measurement, and community-reviewed metadata.

Over 12 months, we will harden and extend support for learnr, learnr2, and Quarto Live, index 5–10 existing tutorials, and use four small examples to test the adapters across teaching contexts. Carpentries-style materials will also be indexed, with stated limits on automated checks. Existing tools will provide browser execution and static hosting. We request USD 10,000 for this work.

Signatories and community support

Shaurita D. Hutchins will lead the project in collaboration with Samuel Bharti. Both are PhD candidates at the University of Alabama at Birmingham.

Hutchins will lead educational design and community development. She will develop the metadata schema, coordinate example tutorials, and establish accessibility and contribution guidance. A computational biologist trained in genetics, genomics, and bioinformatics, she brings experience in Carpentries instruction and reproducible R/Bioconductor workflows. She received a grant to develop [Prioritizing Candidate Genetic Variants Using GTE_x, HuBMAP, and IDG](#) for the Common Fund Data Ecosystem (CFDE) Training Center. Built with Quarto Live, the module combines coding activities that run in the browser with knowledge checks.

Bharti will lead software architecture and engine integration. He will harden and extend adapters that use existing engine checks and produce validation reports in a common format. A former Posit intern, he brings experience developing Shiny applications and R packages, including bioclients, biohttp, and biobouncer. His teaching work includes [r-for-beginners](#), an adaptation of Software Carpentry lessons with accompanying slides. He also maintains automated workflows for publishing package documentation through GitHub Actions and GitHub Pages.

We will jointly design the package API and registry, and share responsibility for governance and the example tutorials.

We requested public feedback through [r/Stats](#) and the [learnr webR discussion](#). Community contributor [@coatless](#) pointed us to [learnr2](#) and Quarto Live. That feedback helped us focus on shared metadata and checks, though the proposed schema still needs review. We will seek engine maintainers' feedback on the schema and the checks each adapter can support.

We will also consult Carpentries and Bioconductor educators, along with university and community instructors. We will add documented feedback and letters of support as received.

Problem

Learners need tutorials that match their goals and current knowledge; instructors need lessons they can assess and reuse in their teaching. Both need clear information about content, prerequisites, difficulty, and duration. They also need evidence that the exercises still work, since a tutorial may remain online after they break or its author stops maintaining it.

The tools used to build a lesson also affect how it runs. [learnr](#) runs tutorials as Shiny applications ([Aden-Buie et al. 2025](#)). [learnr2](#) builds on Quarto Live, which uses webR to execute R in the browser ([learnr2](#), n.d.; [Stagg 2026](#)). [tutorialverse](#) will give instructors and learners a common way to compare tutorial descriptions and check results across these engines.

Evidence of need and prior work

Developing Quarto exercises, [Handel \(2025\)](#) compared [learnr](#), [webexercises](#), [checkdown](#), [exams](#), [quarto-webr](#), and [quarto-live](#) for hosting, maintenance, and exercise support. [Kranz's comparison of RTutor and learnr](#) documents similar tradeoffs in 2019, before webR. Reports from the [learnr](#) community illustrate practical challenges with [custom Shiny questions](#) and [Python exercises that work locally but fail after deployment](#), without establishing their prevalence.

Discovery initiatives address another part of the problem. [rstats-ed](#) compiled a community-curated list of R learning materials, acknowledging its incomplete coverage. [Glitr.org](#) supports discovery and comparison of teaching repositories but excludes educational-quality evaluation ([Geest et al. 2024](#)). [TeSS](#) aggregates training metadata, while [Bioschemas](#) standardizes resource descriptions ([Garcia et al. 2020](#); [Castro et al. 2023](#)).

The Galaxy Training Network combines automated checks with community review and maintenance ([Hiltemann et al. 2023](#)). The [Journal of Open Source Education \(JOSE\)](#) provides formal peer review for reusable educational materials and software. [tutorialverse](#) can record this external review alongside its technical checks.

[tutorialverse](#) will connect these approaches across R tutorial engines through shared descriptions, validation reports, and review and maintenance records. Existing engines will execute tutorials. New engines, browser runtimes, and learning-management systems remain outside scope; static hosting is a deployment mode to describe and check.

Proposal

The five milestones follow a tutorial from description through checking, discovery, and review.

Milestone 1: A minimal metadata schema

We will begin with eight required fields: identifier, title, source URL, lesson URL, engine or format, maintainer, license, and learning objectives. We will map these fields to Schema.org and the Bioschemas TrainingMaterial profile where applicable. Topic, level, prerequisites, packages, deployment mode, duration, accessibility information, and software versions will be optional in the initial schema. Optional review references and DOIs will identify external reviews and publications. The checking tools will generate validation dates and results.

To reduce manual work, we will evaluate drafting metadata from existing tutorial sources. Suggested topics, prerequisites, learning objectives, and difficulty will require human confirmation before publication. Records will distinguish author-supplied values, mechanically extracted information, and inferred suggestions, with review status recorded separately. Parsing, heuristics, or AI may assist drafting; the registry pilot will measure whether it reduces author effort.

Authors will supply missing fields in a small accompanying file without rewriting lessons. Unknown values will remain explicit, and nominations with unknown maintainers or licenses will remain pending review. Adapters will run checks only when the required information is available; unconfirmed suggestions will not establish prerequisites for a check.

We will align the prototype with this draft and invite engine maintainers and educators to review it. Their feedback and the pilot's measures of author effort will guide revisions before we publish version 1.0 with example records and adoption guidance.

Milestone 2: Harden and extend engine adapters

The prototype implements `learnr_adapter()`, `learnr2_adapter()`, and `quarto_live_adapter()`, producing a common report through `check_tutorial()`. We will harden and extend these adapters, refine their interfaces with maintainers, and document supported versions and features. Because `learnr2` and `Quarto Live` share formats, format detection alone cannot always distinguish them.

Adapters may invoke an engine's existing build or checking tools; the engine remains responsible for execution. Conversion between engines is outside the adapters' scope. A capability table will show each adapter's support for metadata import, rendering checks, code execution, and accessibility checks. Engines can support different checks while reporting results in the same format. Carpentries-style materials can be indexed after metadata and link checks. Further checks will depend on the lesson's build system, with those limits stated in the registry.

Milestone 3: Validation across engines

`check_tutorial()` already combines metadata, URL, and optional rendering checks using four statuses: `pass`, `fail`, `not_run`, and `unsupported`. Grant work will add deeper engine-specific checks of required files, declared dependencies, package versions, interactive execution, and exercise identifiers where supported. Basic automated accessibility checks will accompany manual keyboard checks. Reports will distinguish observed results from declared accessibility information.

We will automate reproducible checks and investigate browser tests of representative exercises and assessments. Successful rendering will remain distinct from evidence that interactive behavior works. Reports will identify failing fields or exercises and record the test environment, source version, and date. Automated checks alone will not establish teaching quality or full accessibility.

We will publish a reusable GitHub Actions workflow and integration guidance so authors can run checks in their own repositories without joining the registry. CI will test the package on Linux, macOS, and Windows, using examples with known faults to verify that the checks detect problems. Documented real renders currently cover macOS only.

Milestone 4: A searchable tutorial registry

The registry will link tutorial descriptions to the original repositories and published lessons. Authors will retain responsibility for content and hosting. Automated builds will use YAML entries to create the searchable website and publish `index.json`. A proposed `tutorial_search()` function will return results as an R data frame.

Readers will filter by topic, package, level, learning objective, engine, and deployment mode when those fields are available. Each entry will show duration and maintenance status when known, and distinguish metadata validity from execution checks and human review. Scheduled checks will flag broken links and outdated validation records.

The indexing pilot will cover 5–10 existing tutorials across the supported formats. We will record missing fields and manual corrections to measure author effort. Completion will require valid records and working search results; claims about execution will depend on separate adapter tests.

Milestone 5: Review, governance, and example tutorials

Authors will submit entries through structured pull requests; others can nominate tutorials. Drawing on [rOpenSci's review process](#) and Carpentries lesson-review practices ([The Carpentries 2022](#)), automated checks will precede human review of metadata completeness, scope, licensing, maintenance information, and declared accessibility. We will record external pedagogical or scholarly reviews separately. Registry entries will distinguish review history from current validation and maintenance status. Governance guidance will assign

maintenance responsibilities and explain how to contact maintainers, correct or remove entries, and archive stale resources.

We will use four small example tutorials to test the adapters: `renv`, package development with `usethis` and `devtools`, an introductory DESeq2 workflow, and a Carpentries R companion. We will choose representative exercises and adapt existing material under compatible licenses. Bharti's `r-for-beginners` lessons and Bioconductor training resources offer starting points.

These examples will test metadata mapping, validation reports, and each engine's checks; the indexing pilot will test how existing tutorials are described and found. The four examples will collectively cover all three adapters, without requiring every tutorial to run under every engine. Tests will include browser interaction where supported and local or server execution where needed. Three or four small integration templates will show authors how to connect existing lessons through their chosen engine.

Project plan

Preliminary work

We have implemented an initial tutorialverse prototype supporting `learnr`, `learnr2`, and Quarto Live. The package validates common tutorial metadata, checks source and published URLs, and uses engine adapters to render trusted local tutorials while producing a common validation report. The [adapter review](#) documents successful renders of one public tutorial per engine on macOS. Interactive exercise execution and accessibility remain explicitly unsupported rather than being inferred from successful rendering. Cross-platform CI, curated registry records, and measurements of author effort remain proposed work.

Schedule and working practices

Period	Work and completion criteria
Months 1–2	Review the eight-field schema, map metadata from existing tutorials into pilot records, establish repositories and governance, and begin the example tutorials.
Months 3–5	Harden the three adapters and extend <code>check_tutorial()</code> . Publish a capability table and common report format. Test valid examples and known failures in Linux, macOS, and Windows CI.
Months 6–8	Build registry ingestion and search. Publish <code>index.json</code> and the R search interface. Run the pilot records through the package and document author effort.
Months 9–10	Pilot nominations and human review with external contributors. Test stale-entry handling and gather usability feedback.
Months 11–12	Complete example tutorial and integration-template checks. Address high-priority feedback and release version 1.0 of the package, specification, and registry with maintenance guidance.

We will work openly on GitHub, using issues and milestones to track progress and pull requests for code review. We will publish monthly updates and milestone reports and invite community feedback. Package tests will use `testthat`, with `renv` to reproduce software environments. We will publish documentation with `pkgdown` and distribute the package through R-universe where appropriate.

Code will use the MIT license. Newly authored content and documentation will use CC BY 4.0 where compatible with source obligations; adaptations will retain required attribution and licensing. We will share release announcements and integration guidance with R education communities.

Budget and funding

Requested budget: USD 10,000. This funds 200 hours at USD 50/hour, allocated 60% to project lead Hutchins (120 hours; USD 6,000) and 40% to Bharti (80 hours; USD 4,000). No hardware, cloud, travel, or fee costs are included.

Milestone	Hutchins (h)	Bharti (h)	Total (h)	USD
Minimal schema	18	12	30	1,500
Adapters	30	20	50	2,500
Validation package	33	22	55	2,750
Registry	21	14	35	1,750
Review, governance, and examples	18	12	30	1,500
Total	120	80	200	10,000

If funding is reduced, the schema and validation tools can be used independently of the registry.

Risks and maintenance

The prototype demonstrates the adapter model; adoption remains the main uncertainty. Authors may find the schema burdensome if it duplicates metadata they already maintain. The next experiment is the 5–10-tutorial registry pilot: we will record which fields are reused, supplied manually, or unknown, and measure author effort. These findings and early consultation will guide revisions before version 1.0.

Engine changes could break adapters or expose differences in test coverage. We will document supported versions and browser or server requirements, and use the example tutorials to detect regressions.

We will document contribution and governance procedures so others can maintain the registry without relying on us for every change. Entries will name maintainers and retain source links, validation dates, and status. Scheduled checks and an archival policy will help contributors address stale resources.

Success

Completion requires version 1.0 of the metadata schema, R package, and searchable registry. Deliverables will include three hardened adapters tested in Linux, macOS, and Windows CI; 5–10 indexed tutorials; and four example tutorials tested in CI, collectively covering all three adapters. We will publish three or four integration templates and at least one reusable validation workflow.

At least three external contributors will review the schema, adapters, or tutorials. The registry pilot will test nomination, review, and stale-entry procedures. We will document check results and reviewer feedback, track external submissions, and measure author effort, including whether assisted metadata drafting reduces manual work.

Future work could support additional engines and educational-catalog integrations. Multilingual metadata, richer accessibility checks, and instructor collections remain outside the funded scope.

References

- Aden-Buie, Garrick, Barret Schloerke, J. J. Allaire, and Alexander Rossell Hayes. 2025. *learnr: Interactive Tutorials for R*. Released. <https://CRAN.R-project.org/package=learnr>.
- Castro, Leyla Jael, Patricia M. Palagi, Niall Beard, Teresa K. Attwood, and Michelle D. Brazas. 2023. “Bioschemas Training Profiles: A Set of Specifications for Standardizing Training Information to Facilitate the Discovery of Training Programs and Resources.” *PLOS Computational Biology* 19 (6): e1011120. <https://doi.org/10.1371/journal.pcbi.1011120>.
- Garcia, Leyla, Bérénice Batut, Melissa L. Burke, et al. 2020. “Ten Simple Rules for Making Training Materials FAIR.” *PLOS Computational Biology* 16 (5): e1007854. <https://doi.org/10.1371/journal.pcbi.1007854>.
- Geest, Geert van, Yann Haefliger, Monique Zahn-Zabal, and Patricia M. Palagi. 2024. “Using Glittr.org to Find, Compare and Re-Use Online Materials for Training and Education.” *PLOS ONE* 19 (12): e0308729. <https://doi.org/10.1371/journal.pone.0308729>.
- Handel, Andreas. 2025. *Quizzes and Exercises for Quarto Websites*. <https://www.andreashandel.com/posts/2025-02-25-exercises-in-quarto/>.
- Hiltemann, Saskia, Helena Rasche, Simon Gladman, et al. 2023. “Galaxy Training: A Powerful Framework for Teaching!” *PLOS Computational Biology* 19 (1): e1010752. <https://doi.org/10.1371/journal.pcbi.1010752>.
- learnr2: Interactive R Tutorials*. n.d. <https://github.com/PPBDS/learnr2>.

- Stagg, George. 2026. *quarto-live: WebAssembly Powered Code Blocks and Exercises for R and Python in Quarto Documents*. Released. <https://github.com/r-wasm/quarto-live>.
- The Carpentries. 2022. *Piloting Open Peer Review of Lessons*. <https://carpentries.org/blog/2022/02/carpentries-lab-lesson-review-pilot/>.